

The Dodo Programming Language

Language specification 0.1

Edition: 10 September 2026

Status: Proposed language; normative design specification

A small language. Explicit control. Checked borrowing.

Dodo is a small, ahead-of-time compiled systems programming language with Go-like blocks, Rust-like function signatures, checked borrowing, and explicit hardware access. It targets native applications and bare-metal firmware without requiring a garbage collector, heap, operating system, or scheduler.

Status and interpretation

This edition incorporates the September 2026 ergonomics revisions to the Dodo 0.1 design: consistent name-first declarations, shorter signatures and literals, local type inference, composable constants, value-producing blocks, ranges, and subslices. Earlier type-first declarations and fully explicit forms remain accepted for compatibility. The ownership categories are unchanged.

Must, must not, shall, and shall not express requirements. May expresses permission. Should expresses a recommendation. Unspecified means version 0.1 does not settle a behavior or syntax rule; portable programs must not depend on one implementation's choice.

The specification defines an intended source-language model. It does not assert that an implementation is complete, sound, memory-safe, mature, or secure. Examples explain the design; they are not evidence of compiler conformance. Compiler implementation status belongs in separate project documentation.

Version 0.1 does not completely define lexical rules, expression grammar, ABI, standard library, target object formats, or a formal memory-safety argument. The resolved ergonomics rules are specified below; Appendix C collects the remaining open design items.

Contents

1. Language overview
2. Source files, packages, and visibility
3. Lexical structure
4. Types and values
5. Bindings and initialization
6. Functions and methods
7. Ownership and moves
8. Borrowing and reference validity
9. Destruction and cleanup
10. Expressions and conversions
11. Statements and control flow
12. Results and error propagation
13. Unsafe code
14. Raw-memory validity
15. Core memory and hardware primitives
16. Foreign interfaces, layout, and target attributes
17. Traps and arithmetic failure
18. Diagnostics and implementation obligations
19. Worked examples
- A. Partial grammar
- B. Conformance checklist
- C. Open specification items
- D. Informative references

1. Language overview

1.1. Design profile

Dodo is an ahead-of-time compiled systems language with explicit control flow and checked ownership and borrowing. Ordinary Dodo code shall not require a garbage collector, heap allocator, operating system, or language-level scheduler.

The language supports a design for both hosted native applications and freestanding firmware. Allocation, containers, device drivers, synchronization, and scheduling belong in libraries, rather than mandatory runtime services.

1.2. Primary rules

- Mutation is explicit. Ordinary local bindings are mutable unless their declaration form constrains them.
- Aggregates have a single owner by default. Non-copy aggregates move; they do not implicitly deep-copy.
- Checked references are non-null and statically borrow-checked.
- Overlapping memory permits multiple shared readers or one mutable writer, never both while those accesses are live.
- Errors are ordinary Result values. A call site marks propagation with ?.
- Explicit unsafe syntax contains unchecked operations.
- Explicit core-library primitives provide hardware access; ordinary mutable references must not be fabricated for device memory.
- Cleanup is deterministic on normal scope exits. Traps abort without guaranteed unwinding or cleanup.

1.3. Deliberate omissions

Version 0.1 has no operator or function overloading, inheritance, implicit constructors, exceptions, macros, reflection, closures, async/await, or language-level threads. It has no user-defined implicit conversions, iterator protocol, specialization, or metaprogramming facilities.

Operators are compiler-defined for primitive types and supported simple enums. Custom behavior uses named methods.

2. Source files, packages, and visibility

2.1. Packages

A source unit declares its package:

```
package samples
```

The relationship between source files and package compilation units is unspecified in version 0.1.

2.2. Imports

Packages are imported by string path. Imported declarations use qualified names, such as `mmio.write32`:

```
import "core/mmio"
```

2.3. Visibility

Declarations and fields are package-private unless marked `pub`. Letter case has no visibility meaning. A public API must not expose a private type. A public enum exposes its variants.

Re-exports, import aliases, wildcard imports, cyclic packages, and package initialization order are unspecified.

3. Lexical structure

3.1. Statements and blocks

Newlines normally terminate statements. Semicolons separate the clauses of a three-part for statement. Blocks require braces; conditions do not require parentheses.

```
for i := 0; i < n; i += 1 {  
    // body  
}
```

3.2. Comments

`//` begins a line comment that extends to the end of the line. Block comments, nested comments, documentation comments, and the exact effect of comments on lexical disambiguation are unspecified.

3.3. Identifiers and keywords

Examples use identifiers such as `parse_byte`, `Samples`, and `set_address`. The design uses these syntactic or contextual keywords:

```
package import pub fn struct enum const return if else for in  
break continue match unsafe extern as from static Self self void
```

The exact identifier character set, Unicode normalization, case sensitivity, reserved-word policy, and raw-identifier syntax are unspecified. Implementations should reject source whose meaning depends on an undocumented lexical rule. This list is a record of the supplied design, not a complete lexical grammar.

3.4. Literals

- Integer literals are constrained by context. An unconstrained integer literal defaults to `isize`.
- Typed integer literals include forms such as `0u32`.
- Byte literals include forms such as `b'a'`.
- String literals, such as `"hi"`, refer to immutable program-lifetime storage.
- Byte strings, such as `b"hi"`, also refer to immutable program-lifetime storage.
- Array literals use `[1, 2, 3]`, with the length inferred from the elements.

- Repeated array literals use `[value; N]`, where `N` is an integer constant expression.

Numeric bases, digit separators, floating-point literals, character and string escapes, raw strings, Boolean literal spelling, and source encoding are not fully specified.

4. Types and values

4.1. Built-in types

Type	Meaning
<code>bool</code>	Boolean value
<code>i8, i16, i32, i64</code>	Signed fixed-width integers
<code>u8, u16, u32, u64</code>	Unsigned fixed-width integers
<code>isize, usize</code>	Pointer-sized signed and unsigned integers
<code>f32, f64</code>	Floating-point values
<code>void</code>	No return value
<code>[N]T</code>	Owned array of exactly <code>N</code> elements of <code>T</code>
<code>&[T]</code>	Shared slice
<code>&mut [T]</code>	Mutable slice
<code>&T</code>	Checked, non-null shared reference
<code>&mut T</code>	Checked, non-null mutable reference
<code>*const T</code>	Unchecked, nullable raw pointer for immutable access
<code>*mut T</code>	Unchecked, nullable raw pointer for mutable access
<code>&str</code>	UTF-8 string view
<code>Result<T, E></code>	Success or error value
<code>Option<T></code>	Optional value, conceptually <code>some(T)</code> or <code>none</code>

4.2. Arrays and slices

An array `[N]T` owns exactly `N` elements. `N` is a nonnegative integer constant expression, including references to named constants. Array literals infer their length and take their element type from context, typed elements, or the ordinary literal defaults:

```
values: [4]u16 = [1, 2, 3, 4]
bytes := [0u8; 256]
```

`[value; N]` evaluates `value` exactly once, including when `N` is zero, and repeats it `N` times. The element must be copyable; repetition never clones owned values or duplicates mutable references. Empty lists need an element type from context. The explicit composite form `[4]u16{1, 2, 3, 4}` remains accepted.

Arrays and slices expose `.len`. Borrowing an array may coerce to a slice without allocation. `&values[start..end]` produces a shared subslice and `&mut values[start..end]` produces an exclusive mutable subslice. A bare `values[start..end]` also produces a shared subslice. Missing start and end

bounds mean zero and the collection length respectively. The source, start, and end are evaluated once, in that order. Bounds must satisfy $0 \leq \text{start} \leq \text{end} \leq \text{length}$; violations trap in every optimization profile. An empty slice at the end of a collection is valid. Bounds may read the captured source (for example, `data.len`) but must not move or mutate it; mutable access is established after the bounds have been evaluated.

Subslices retain their source's checked lifetime. Mutable slicing requires exclusive access; ranges do not establish disjointness between simultaneous mutable slices. UTF-8 string slicing is not provided by this syntax.

4.3. Checked references

`&T` permits shared reading. `&mut T` permits mutation and grants exclusive access to the referenced region for the loan's duration. Both are checked and non-null.

Field access and indexing automatically dereference checked references. Raw pointers are never automatically dereferenced.

4.4. Raw pointers

`*const T` and `*mut T` are unchecked and nullable. Copying or storing a raw pointer does not dereference memory. Raw-pointer dereference, pointer arithmetic, and conversion from a raw pointer to a checked reference require `unsafe`.

4.5. Structs

Structs contain named fields and use named-field literals. Fields remain package-private unless individually marked `pub`.

```
pub struct Pin {
    set_address: usize
    clear_address: usize
    mask: u32
}

pin := Pin{
    set_address: set,
    clear_address: clear,
    mask,
}
```

A literal field written as `mask` is shorthand for `mask: mask`. It reads or moves the binding exactly as the explicit initializer does.

4.6. Enums

Enums are plain tagged values or tagged values with payloads. A public enum exposes its variants. Matching may destructure payloads.

```
pub enum ParseError { Length, Digit }

// Conceptual payload form from the design:
// Invalid(byte: u8)
```

The conceptual payload example does not establish a complete payload grammar.

4.7. Generics

Basic type-parameter generics use angle brackets, as in `Name<T>`. Every instantiation must satisfy ordinary type and ownership checks. There are no traits, specialization, user-defined implicit conversions, or metaprogramming.

Function calls and struct literals infer omitted type arguments from their arguments or fields and an expected result type. For example, `identity(42i32)` and `Box{value: 42i32}` infer `i32`. An expected type also constrains unsuffixed literals, as in `value: i32 = identity(42)`. Inference is local; it does not infer function signatures from bodies or solve types from later statements. Conflicting or unresolved type arguments require a diagnostic. Explicit arguments remain available, such as `identity<i32>(42)`.

`some(value)` infers `Option<T>` from its payload when no expected type exists. `none`, `ok`, and `err` still need enough context to determine their full types. Generic constraints, variance, and separately compiled interfaces remain open.

5. Bindings and initialization

5.1. Local bindings

`:=` declares a mutable local whose type follows from its initializer and surrounding constraints. Explicit declarations use `name: Type`, consistently with parameters, fields, and enum payloads:

```
name := value
count: u32 = 0
```

5.2. Constants

`const` declares a compile-time constant:

```
const LIMIT: u32 = 64
```

Constants may refer to other constants, including imported public constants and previously declared local constants. Package constants may refer forward; cycles are rejected. Arithmetic uses the declared types and selected target width. Runtime variables and mutable static storage cannot determine a constant or array length. Compile-time function calls are not part of this revision.

5.3. Definite initialization

Every read requires definite initialization. A moved-from binding is not initialized for reading until it is reinitialized. The implementation must reject a read unless the binding or subobject is established as initialized on every applicable control-flow path.

6. Functions and methods

6.1. Functions

An omitted return type means `void`; `fn reset() {}` and `fn reset() -> void {}` have the same signature. Functions returning a value must declare its type. Function returns remain explicit; a final expression in a function body is not an implicit return.

```
pub fn add(a: u32, b: u32) -> u32 {
    return a + b
}
```

6.2. Arguments

Passing a copy type copies its value. Passing an owned non-copy value transfers ownership. Passing a checked reference passes or reborrows access according to the borrowing rules. Free-function calls make new borrows explicit:

```
update(&mut item)
```

Reference arguments are reborrowed when needed.

6.3. Methods and associated functions

Methods are declared inside their struct. Their first parameter is named self:

Receiver	Effect
&self	Shared read access
&mut self	Exclusive mutable access
self	Consumes the value

The equivalent explicit forms self: &Self, self: &mut Self, and self: Self remain accepted. Shorthand is valid only for struct receivers.

item.update() automatically borrows or reborrows the receiver as needed. A function inside a struct without a receiver is called as Type.name(...). Dispatch is static. There is no inheritance or separate implementation block.

6.4. Borrowed returns

A function returning a checked borrow must identify the source that bounds the result's lifetime:

1. A borrowed receiver is the inferred source when present.
2. Otherwise, exactly one borrow-carrying parameter is the inferred source.
3. An ambiguous signature declares its sources with from(...).
4. A borrow with no input source requires from(static) and program-lifetime storage.

```
fn choose(a: &u8, b: &u8, first: bool) -> &u8 from(a, b) {
    // body
}
```

The result conservatively keeps every named source borrowed. The compiler must check the function body against its declared or inferred contract. A contract does not extend a lifetime and never permits a reference to a local destroyed on return. Source-based contracts replace user-written lifetime parameters, not lifetime checking.

7. Ownership and moves

7.1. Copy categories

Category	Types
Copy by default	Scalars, shared references, raw pointers
Move by default	Structs, arrays, tagged values, mutable references

Passing or assigning an owned move value transfers ownership. A moved-from binding cannot be read until reinitialized. Duplicating an aggregate requires an explicit ordinary function, such as `clone`; there is no implicit deep copy.

7.2. Subobjects

Separate struct fields may be borrowed independently. Potentially overlapping indexed accesses are rejected unless a checked operation, such as slice splitting, establishes disjointness.

Partial moves are otherwise unspecified, except that moving fields out of a value with a custom drop method is forbidden.

8. Borrowing and reference validity

8.1. Aliasing

For overlapping memory, a program may have multiple live shared readers or one live mutable writer, never both. While a loan is live, its owner cannot move, be destroyed, or access the borrowed region incompatibly. Safe code cannot convert a shared reference to a mutable reference.

8.2. Loan extent

Borrowing is checked at compile time; it does not require a runtime reference counter. A loan ends after its last possible use on each control-flow path, including any destructor that could access it. Consequently, a loan can end before its enclosing lexical block if no later path can use it.

8.3. Reborrowing

Reborrowing a mutable reference temporarily suspends the original reference for the duration of the reborrow. It does not duplicate exclusive access.

8.4. Borrow-carrying values

Structs may contain references. A value carrying borrows has one conservative inferred lifetime bounded by every retained source. Result, Option, and containers propagate this dependency.

Replacing a borrowed field cannot extend the value's inferred lifetime. Independently varying field lifetimes and self-referential owning structs are excluded from version 0.1.

8.5. Disjoint indexing

Potentially overlapping indexed accesses are rejected unless a checked operation establishes disjointness. The language shall not silently add copying, allocation, or runtime borrow checks to accept a conflict.

8.6. Safety scope

Memory safety is a goal for safe code, conditional on a sound compiler, a sound core library, and correct unsafe and FFI contracts. It is not a general security guarantee.

9. Destruction and cleanup

9.1. Normal exits and replacement

Owned values are destroyed in reverse declaration order on ordinary scope exits, including return, `?`, break, and continue. Moved values are not destroyed twice. Overwriting an initialized owned value destroys the old value before replacing it.

9.2. Custom destruction

A struct may declare the reserved method:

```
fn drop(&mut self) {  
    // cleanup  
}
```

The method runs before the fields are destroyed. It cannot return an error or be called directly. `core.drop(value)` consumes and destroys a value early. Moving fields out of a value with a custom drop method is forbidden.

9.3. Traps

Traps abort without unwinding or guaranteed cleanup. Programs must not depend on cleanup running in response to a trap.

10. Expressions and conversions

10.1. Access and calls

Operation	Syntax
Function call	<code>f(args)</code>
Method call	<code>receiver.method(args)</code>
Field access	<code>value.field</code>
Indexing	<code>value[index]</code>
Shared borrow	<code>&value</code>
Mutable borrow	<code>&mut value</code>
Explicit dereference	<code>*value</code>

Field access and indexing automatically dereference checked references, never raw pointers. An explicit dereference obtains a scalar value where needed.

10.2. Numeric conversion

as performs an explicit numeric conversion, checked where needed. Named operations must express wrapping or truncation.

```
weight := i as u32 + 1
total += weight * (*value as u32)
```

10.3. Operators

Examples establish arithmetic, comparison, Boolean conjunction, shifts, bitwise OR, assignment, and compound assignment. Operators are compiler-defined for primitive types and supported simple enums; user code cannot overload them.

The complete operator set, precedence, associativity, evaluation order, short-circuit details, and mixed-width arithmetic rules are unspecified. Portable code must not rely on an unstated ordering rule.

11. Statements and control flow

11.1. Blocks and conditions

A block is a brace-delimited sequence of statements. Control-flow constructs that take blocks require braces. Conditions have type `bool`; numeric truthiness is not specified. Parentheses are optional around conditions.

```
if condition {
    // ...
} else if other {
    // ...
} else {
    // ...
}
```

`if`, `match`, `unsafe`, and `plain` blocks can also produce values in expression positions. The final expression in a value block supplies its value:

```
limit := if fast { 100u32 } else { 10u32 }
value := unsafe { ptr.read(pointer) }
```

Every continuing path must supply a value of the same type, and at least one value-producing path must exist. Other paths may return, propagate an error, `break`, or `continue` as permitted by their enclosing function and loops. Only the selected branches execute. A value is transferred out before the block's locals are destroyed in reverse order. Borrows of the block's local storage cannot escape. Newlines and semicolons remain statement separators inside value blocks.

11.2. Loops

`for` is the only loop keyword. `break` and `continue` apply to the nearest enclosing loop.

```
for { ... }
for condition { ... }
for i := 0; i < n; i += 1 { ... }
for i in 0..n { ... }
for _ in 0..n { ... }
for item in items { ... }
```

```
for index, item in items { ... }
for item in &mut items { ... }
for index, item in &mut items { ... }
```

A range loop `for i in start..end` visits integers from `start` up to, excluding, `end`. Its integer bounds have the same type, inferred from typed bounds or the ordinary integer default. Both bounds are evaluated once, left to right, before the loop. A reversed or empty range performs zero iterations. Each iteration gets a fresh binding; assigning to it does not alter the loop counter. `_` discards the iteration value. `break` and `continue` keep their ordinary cleanup semantics. Ranges are built-in loop syntax, without an iterator protocol or a first-class range type.

Collection `foreach` supports arrays and slices. It evaluates its input once, borrows rather than consumes the collection, and creates fresh bindings on each iteration. Indexed forms bind a `usize` index. Shared iteration binds `&T` elements; mutable iteration binds `&mut T` elements.

The collection stays borrowed for the iteration. A mutable element loan cannot escape its iteration in version 0.1. There is no iterator protocol and no `while`, `loop`, or `do` keyword.

11.3. Matching

`match` is exhaustive and has no fallthrough. In statement position its arms contain blocks:

```
match value {
  pattern => { ... }
  pattern => { ... }
}
```

In expression position arms may contain a single expression or a value block:

```
return match parsed {
  ok(value) => value,
  err(_) => fallback,
}
```

Patterns include literals, enum variants, payload bindings, and `_`. Variant names may omit the enum qualifier when matching a value of that enum type. Matching an owned non-copy value consumes it. Matching `&value` or `&mut value` borrows its payloads.

Guards, nested structural patterns beyond enum payloads, or-patterns, ranges, integer-domain exhaustiveness, and further binding modes are unspecified.

12. Results and error propagation

12.1. Result values

`!E` is shorthand for `Result<T, E>`. It is not an exception type and does not imply a separate ABI. `!` binds outside reference and slice type syntax.

The prelude supplies `ok(value)` and `err(error)`. A fallible function must return one of these explicitly. A `void!E` function succeeds with `ok()`.

12.2. Propagation

A trailing `?` on a call result produces the success value or immediately returns the same error from the enclosing function:

```
high := nibble(text[0])?
low  := nibble(text[1])?
```

`?` is permitted only when the enclosing function returns a `Result` with the same error type. It performs no implicit conversion. Cleanup follows the normal return rules.

12.3. Local recovery

`match` handles failure locally. There are no exception classes, error traits, implicit error conversions, or configurable propagation operators. Error translation explicitly matches a source error and constructs a destination error.

12.4. Mandatory handling

Discarding a `Result`, including through `_ =`, is a compile-time error. A program must forward it, propagate it, or explicitly handle both variants with `match`. An intentionally empty error arm counts as handling.

13. Unsafe code

13.1. Unsafe blocks

`unsafe` introduces a lexical block permitting particular unchecked operations:

```
unsafe {
    mmio.write32(self.set_address, self.mask)
}
```

The following operations require unsafe context:

- Raw-pointer dereference and arithmetic.
- Raw-pointer conversion to a checked reference.
- Foreign function calls.
- Unchecked memory access.
- Inline assembly.

An unsafe block does not disable type checking, ordinary borrowing rules, or safe indexing checks.

13.2. Unsafe functions and wrappers

An unsafe function places safety obligations on its caller:

```
pub unsafe fn claim(set: usize, clear: usize, mask: u32) -> Pin {
    // ...
}
```

Its body still requires explicit unsafe blocks around unchecked operations. Public unsafe functions must document their preconditions. Each unsafe block should explain why those preconditions hold at the operation site.

A safe wrapper must enforce the underlying unsafe preconditions for every safe caller allowed by its public API, not merely one expected caller.

14. Raw-memory validity

Raw-pointer operations must respect the pointer's originating allocation or object, including bounds, alignment, initialization, valid representations, and aliasing requirements. Converting a raw pointer to a checked reference also asserts validity for the borrow's checked lifetime and compatibility with every other access.

A cast or unsafe block does not make an invalid checked reference valid.

The formal provenance model, allocation identity, integer-to-pointer semantics, exposed-address model, and foreign-allocation lifetime model remain unspecified.

15. Core memory and hardware primitives

15.1. Pointers and memory

`core.ptr` provides raw reads and writes, offsets, unaligned access, and explicitly unchecked operations. `core.mem` supplies at least `size_of`, `align_of`, `offset_of`, and `MaybeUninit<T>`. Exposing uninitialized storage as `T` is unsafe.

Allocation is optional and explicit. Recoverable allocation failure returns `Result`.

15.2. Memory-mapped I/O

`core.mmio` supplies width-specific volatile operations, including forms equivalent to:

```
unsafe fn read32(address: usize) -> u32
unsafe fn write32(address: usize, value: u32) -> void
```

These operations address device memory directly without fabricating an ordinary mutable reference. Only access widths supported by the target are accepted. Unsupported widths must not silently become multiple narrower accesses.

Valid addresses, permissions, device side effects, and configuration are unsafe obligations of the caller.

15.3. Volatile operations

Volatile operations must not be removed or coalesced. They must retain compiler-level ordering with other volatile operations. They are not atomics, CPU barriers, or synchronization operations.

Interrupts, DMA, and multiple cores require appropriate atomics, target barriers, or correctly scoped critical sections.

15.4. Mutable static state

Mutable static globals require unsafe access unless a synchronization primitive encapsulates them. Core atomics are the narrow exception to ordinary immutable shared-access restrictions; they must not expose ordinary mutable references to storage accessible concurrently.

Interrupt-safe wrappers must account for every context that can reach the underlying storage.

15.5. DMA

A DMA wrapper must keep its transfer buffer alive and unavailable for conflicting CPU access until completion. Dropping an active transfer must safely stop it or retain the buffer until the device stops accessing it.

A move-only peripheral handle prevents accidental language-level duplication, but does not establish hardware exclusivity. The board or platform layer must establish that property.

16. Foreign interfaces, layout, and target attributes

16.1. C ABI

A C-ABI declaration uses syntax such as:

```
unsafe extern "C" fn send(data: *const u8, length: usize) -> i32
```

Correct C types and declarations for the target remain the binding author's responsibility.

16.2. Layout and attributes

@repr(C) requests C-compatible struct layout. A closed compiler-defined set of attributes handles alignment, sections, exported symbols, and target interrupt entry points. Attributes are not programmable macros.

16.3. Assembly and barriers

Inline assembly and target memory barriers are target intrinsics. Unaligned data requires explicit unaligned operations; ordinary references with invalid alignment must not be created.

The complete attribute set and grammar, object-file symbol model, C type mapping, enum layout, padding, non-C field reordering, and interrupt ABI syntax are unspecified.

17. Traps and arithmetic failure

Safe integer overflow, division by zero, invalid shifts, and out-of-bounds indexing trap in every build profile. Numeric as conversions are explicit and checked where needed. Wrapping and truncation require named operations. Bounds checks may be removed only when proven unnecessary.

A freestanding target supplies entry and linker configuration and a non-returning panic handler. Abort does not unwind. The language guarantees neither real-time deadlines nor hardware correctness.

The concrete trap mechanism, panic-handler signature, diagnostic payload, and platform action, such as process termination or target reset, are target-specific and not fully specified.

18. Diagnostics and implementation obligations

Borrow-conflict diagnostics should identify:

- The conflicting access.
- The loan's origin.
- The use that keeps the loan live.

- A safe repair direction, such as ending the loan sooner or separating disjoint data.

Diagnostics should not propose unsafe code as the default repair.

Before an implementation is described as memory-safe, its design and tests must cover reference aliasing, raw-pointer origins, mutation of borrowed fields, generic lifetime propagation, destruction, and interrupt/DMA interactions. This proposed borrowing model is not a soundness proof.

19. Worked examples

19.1. Structs, methods, and borrowing

This example demonstrates mutable and indexed foreach, method receivers, and a return borrow inferred from self, without allocation or explicit lifetime parameters.

```
package samples

pub struct Samples {
  values: [4]u16

  pub fn offset(&mut self, amount: u16) {
    for value in &mut self.values {
      *value += amount
    }
  }

  pub fn weighted_sum(&self) -> u32 {
    total := 0u32
    for i, value in self.values {
      weight := i as u32 + 1
      total += weight * (*value as u32)
    }
    return total
  }

  pub fn view(&self) -> &[u16] {
    return &self.values
  }
}

pub fn demo() -> u32 {
  samples := Samples{
    values: [1, 2, 3, 4],
  }

  view := samples.view()
  first := view[0]
  samples.offset(10)
  return samples.weighted_sum() + first as u32
}
```

`view()` returns a slice tied to `self`. Copying the scalar `view[0]` is the last use of the view, allowing the loan to end before `offset(10)`. A later use of `view` would keep the shared loan live and require rejection of the mutation. Moving `samples` while its view remains live must also be rejected.

`demo()` evaluates to 131. In `offset`, `value` is `&mut u16`. In `weighted_sum`, `value` is `&u16` and `i` is `usize`.

19.2. Result propagation and recovery

```
package hex

pub enum ParseError { Length, Digit }

fn nibble(ch: u8) -> u8!ParseError {
  if ch >= b'0' && ch <= b'9' {
    return ok(ch - b'0')
  } else if ch >= b'a' && ch <= b'f' {
    return ok(ch - b'a' + 10)
  } else {
    return err(ParseError.Digit)
  }
}

pub fn parse_byte(text: &[u8]) -> u8!ParseError {
  if text.len != 2 {
    return err(ParseError.Length)
  }
  high := nibble(text[0])?
  low := nibble(text[1])?
  return ok((high << 4) | low)
}

pub fn parse_or(text: &[u8], fallback: u8) -> u8 {
  return match parse_byte(text) {
    ok(value) => value,
    err(_) => fallback,
  }
}
```

`u8!ParseError` means `Result<u8, ParseError>`. `parse_or(b"2a", 0)` returns 42; `parse_or(b"zz", 7)` returns 7. Calls fail through declared result values or non-recoverable traps. `?` marks propagation and possible early return; `match` marks local recovery.

19.3. An unsafe hardware boundary

```
package gpio
import "core/mmio"

pub struct Pin {
  set_address: usize
  clear_address: usize
  mask: u32

  // SAFETY: The caller supplies valid, aligned 32-bit SET/CLEAR registers,
  // a valid mask, and exclusive peripheral access for the handle's lifetime.
  // The peripheral must remain powered and configured while the handle exists.
  pub unsafe fn claim(set: usize, clear: usize, mask: u32) -> Pin {
    return Pin{set_address: set, clear_address: clear, mask}
  }

  pub fn high(&mut self) {
    // SAFETY: claim's contract guarantees a valid exclusive SET register.
    unsafe {
      mmio.write32(self.set_address, self.mask)
    }
  }
}
```

```

    pub fn low(&mut self) {
        // SAFETY: claim's contract guarantees a valid exclusive CLEAR register.
        unsafe {
            mmio.write32(self.clear_address, self.mask)
        }
    }
}

pub fn pulse(pin: &mut Pin, count: usize) {
    for _ in 0..count {
        pin.high()
        pin.low()
    }
}

```

Pin has private fields and moves rather than copies. high and low are safe only if the board layer establishes and preserves claim's contract. The registers are write-only SET/CLEAR aliases. The example does not authorize arbitrary read-modify-write operations or guarantee pulse timing.

Appendix A. Partial grammar

This EBNF-style summary normalizes syntax established by the design. It is intentionally partial: expressions, lexical productions, precedence, and some declaration modifiers remain incomplete. The summary is not a complete parser specification. In particular, the C-ABI prototype example in section 16 has no body, while the draft's general function production below ends in a block.

```

program      = package-decl, { import-decl | declaration } ;
package-decl = "package", identifier, newline ;
import-decl  = "import", string-literal, newline ;

declaration  = [ "pub" ],
               ( function-decl | struct-decl | enum-decl )
               | const-decl ;
const-decl   = "const", identifier, ":", type, "=", expression, newline ;

function-decl = [ "unsafe" ], [ "extern", string-literal ],
               "fn", identifier, [ type-params ], "(", [ parameters ], ")",
               [ "->", type ], [ borrow-source ], block ;
parameters    = parameter, { ",", parameter } ;
parameter     = identifier, ":", type | "self" | "&", [ "mut" ], "self" ;
borrow-source = "from", "(", ( "static" | identifier-list ), ")" ;

struct-decl   = "struct", identifier, [ type-params ], "{",
               { field-decl | function-decl }, "}" ;
field-decl    = [ "pub" ], identifier, ":", type, newline ;
enum-decl     = "enum", identifier, [ type-params ], "{",
               enum-variant, { ",", enum-variant }, "}" ;

type          = primitive-type
               | identifier, [ type-args ]
               | "[", constant-expression, "]", type
               | "&", [ "mut" ], type
               | "&", [ "mut" ], "[", type, "]"
               | "*const", type | "*mut", type
               | type, "!", type ;

block         = "{", { statement }, "}" ;
statement     = local-decl | expression-stmt | return-stmt
               | if-stmt | for-stmt | match-stmt

```

```

local-decl    | break-stmt | continue-stmt | unsafe-block ;
              = identifier, ":", expression, newline
              | identifier, ":", type, [ "=", expression ], newline ;
return-stmt   = "return", [ expression ], newline ;
if-stmt       = "if", expression, block,
              { "else", "if", expression, block },
              [ "else", block ] ;
for-stmt      = "for", block
              | "for", expression, block
              | "for", for-init, ";", expression, ";",
              expression, block
              | "for", identifier, "in", expression, [ "..", expression ], block
              | "for", identifier, ",", identifier,
              "in", expression, block ;
match-stmt    = "match", expression, "{", { match-arm }, "}" ;
match-arm     = pattern, ">=", block ;
unsafe-block  = "unsafe", block ;
array-literal = "[", [ expression, { ",", expression }, [ ",", " ] ], "]"
              | "[", expression, ";", constant-expression, "]" ;
struct-field  = identifier, [ ":", expression ] ;
subslice      = expression, "[", [ expression ], "..", [ expression ], "]" ;
match-expr    = "match", expression, "{", { value-arm }, "}" ;
value-arm     = pattern, ">=", ( expression | value-block ), [ ",", " ] ;
value-block   = "{", { statement }, expression, "}" ;

```

Appendix B. Conformance checklist

A compiler claiming Dodo 0.1 conformance should, at minimum:

- Accept the specified declarations, functions, methods, control flow, and types.
- Default omitted return types to void and require explicit function returns.
- Infer local generic arguments and array elements where the context suffices.
- Resolve constant dependencies and reject cycles and invalid array lengths.
- Preserve evaluation order, ownership, and cleanup in value blocks and ranges.
- Enforce definite initialization and invalidation after moves.
- Enforce shared-versus-exclusive borrowing for overlapping memory.
- Check borrowed-return contracts and reject references escaping destroyed locals.
- Destroy owned values deterministically on normal exits, including `?`, `break`, and `continue`.
- Reject discarded Result values.
- Restrict unchecked operations to the specified unsafe boundary.
- Trap on safe overflow, division by zero, invalid shifts, and invalid indexing in every build profile.
- Preserve volatile-access semantics.
- Avoid a mandatory garbage collector, heap, scheduler, or exception runtime when the program does not request one.

These requirements alone do not constitute a proof of soundness.

Appendix C. Open specification items

These items remain open in 0.1. A future revision should settle them before independent implementations are expected to agree on output or diagnostics.

1. Lexing: Source encoding, identifier classes, Unicode normalization, whitespace, block comments, escapes, and complete literal grammars.
2. Expressions: Complete operators, precedence, associativity, evaluation order, short-circuit semantics, and compound-assignment desugaring.
3. Primitive representation: Signed-integer representation, floating-point model, Boolean representation, pointer-sized widths per target, and endianness.
4. Enums and Option: All payload forms, discriminants, layout, niche optimizations, and further optional-value operations.
5. Generics: Constraints, inference beyond the local rules in section 4.7, separate compilation, instantiation, variance, and code sharing.
6. Patterns: Complete grammar, binding modes, and exhaustiveness algorithm.
7. Pointers: Provenance, integer conversion, foreign-allocation identity, and the alias model governing unsafe raw access.
8. ABI and layout: Non-C structs, enums, calling conventions beyond declared C ABI, symbol naming, target attributes, and interrupt ABI.
9. Runtime integration: Entry convention, panic handler, trap encoding, startup, linker inputs, and hosted runtime interface.
10. Core library: Exact signatures and semantics of pointers, memory, atomics, allocation, slice splitting, barriers, and named primitives.
11. Packages: File mapping, dependencies, aliases, cycles, initialization, naming, and visibility across compilation units.
12. Diagnostics: Required categories and machine-readable stability beyond the borrow-diagnostic recommendations.

Other points explicitly left open in their relevant sections include partial moves, comment disambiguation, raw identifiers, and detailed declaration modifiers. An implementation may document choices for these gaps; such choices must be distinguished from requirements of this source specification.

Appendix D. Informative references

The supplied design cites these influences. They are informative, not normative Dodo rules, and this edition does not incorporate their contents by reference.

- Go specification, “For statements”: <https://go.dev/ref/spec>
- Rust Reference, “Lifetime elision”: <https://doc.rust-lang.org/reference/lifetime-elision.html>
- Rust core::result: <https://doc.rust-lang.org/core/result/>
- Rust Reference, “The unsafe keyword”: <https://doc.rust-lang.org/reference/unsafe-keyword.html>

- Rust core::ptr::read_volatile: https://doc.rust-lang.org/core/ptr/fn.read_volatile.html

End of Dodo Language Specification 0.1.